

RIDEFRIEND

Prompts de Desenvolvimento · v2.0

Actualizado com Arquitectura Multi-Mercado

AO MZ BR CV PT NG

NOVO	ACTUALIZADO	IGUAL
------	-------------	-------

Versão	2.0 — integra arquitectura multi-mercado
Prompts NOVOS	P0 (actualizado), L1, L2, L3
Prompts ACTUALIZADOS	P1, P2, P3, P4, P5, P8, P9, P10, P11
Prompts IGUAIS	P6 (motorista), P7 (mapa)
Total de Prompts	14 (12 base + 3 localização)
Mercados Suportados	AO · MZ · BR · CV · PT · NG

Índice de Prompts — v2.0

Use a coluna ESTADO para saber o que mudou. IGUAL = usar prompt da v1 sem alterações. ACTUALIZADO = usar o prompt deste documento. NOVO = prompt completamente novo, não existia na v1.

Nº	Prompt	Entrega	Período	Estado
P0	Contexto Base	Project Instructions Claude	Antes de tudo	ACTUALIZADO
P1	Setup Inicial	package.json, tsconfig, theme, types, stores + market store	S1 D1-2	ACTUALIZADO
P2	Base de Dados	SQL Supabase: tabelas, RLS, funções + bus_stops + market_code	S1 D2-3	ACTUALIZADO
P3	Autenticação	OTP SMS multi-provider, PhoneInput market-aware, onboarding	S2 D1-2	ACTUALIZADO
P4	Localização	GPS realtime, Haversine, geocoding adaptativo por mercado	S2 D3-5	ACTUALIZADO
PL1	i18n + MarketConfig	Configs 6 mercados, i18n herança, detector automático	S2 D5	NOVO
PL2	Market Onboarding	Selector de mercado, feature flags, RGPD/LGPD por mercado	S3 D1	NOVO
PL3	SMS Multi-Provider	AfricasTalking / Twilio / Termii dispatcher	S3 D1	NOVO
P5	Ecrã Passageiro	HomeScreen, DriverCard, AlertButton — textos via useT()	S3 D2-3	ACTUALIZADO
P6	Ecrã Motorista	DriverHome, RouteInput, detecção passageiros rota	S3 D4-5	IGUAL
P7	Mapa	MapScreen, marcadores animados — provider via market config	S4 D1-2	IGUAL
P8	Notif. + SOS	Push notifications, SOS com emergência do market config	S4 D3-5	ACTUALIZADO
P9	Backend + Deploy	API Node.js, Nginx, PM2, env vars multi-provider	S5	ACTUALIZADO
P10	Rede + Perfil	NetworkScreen, Perfil, MarketSelector nas Definições	S6	ACTUALIZADO
P11	Testes + Beta	Jest, Supertest, seed multi-mercado, docs API	S7	ACTUALIZADO

P0

CONTEXTO BASE DO PROJECTO

Project Instructions

ACTUALIZADO

O que mudou vs v1: Adicionado bloco MULTI-MERCADO com stack i18n, interface MarketConfig, e convenção useT() / useMarket() em todo o código.

```
+ i18next + react-i18next (internacionalização)
+ expo-localization (detecção de locale do dispositivo)
+ Arquitectura multi-mercado: Market Config + i18n namespace + Feature Flags
+ Mercados: ao · mz · br · cv · pt · ng
+ Convenção: useT("namespace") para todos os textos da UI
+ Convenção: useMarket().config para aceder à config do mercado activo
+ Convenção: useMarket().hasFeature("flag") para feature flags
```

PROMPT 0 – colar em: Claude.ai → Project → Instruções do Projecto

És o arquitecto de software principal do projecto RideFriend.
Lembra este contexto em todas as sessões deste projecto.

PRODUTO

RideFriend é uma app móvel (iOS + Android) que conecta passageiros com motoristas da sua rede de confiança – família, amigos, colegas, vizinhos.
Mercado primário: Angola (Luanda). Idioma base: Português de Portugal.

STACK DEFINITIVA

```
Frontend : React Native 0.74 + Expo SDK 51 + TypeScript (strict)
Navegação: React Navigation 6 (Bottom Tabs + Stack)
Estado    : Zustand
DB         : Supabase (PostgreSQL + PostGIS + Realtime)
Auth       : Supabase Auth + OTP SMS (provider por mercado)
Mapas      : react-native-maps + OSM (Google Maps opcional por mercado)
Push       : Expo Notifications + FCM + APNs
Backend    : Node.js 20 + Express + TypeScript + Prisma
Hosting    : Hostinger VPS KVM2 Ubuntu 22.04 + Nginx + PM2
i18n       : i18next + react-i18next + expo-localization
```

MULTI-MERCADO (novo em v2)

A app suporta 6 mercados numa só codebase:

```
ao → Angola      (pt-AO, +244, AfricasTalking, AOA)
mz → Moçambique (pt-MZ, +258, AfricasTalking, MZN)
br → Brasil      (pt-BR, +55, Twilio, BRL, Google Maps, PIX)
cv → Cabo Verde (pt-CV, +238, AfricasTalking, CVE)
pt → Portugal    (pt-PT, +351, Twilio, EUR, RGPD obrigatório)
ng → Nigéria     (en-NG, +234, Termii, NGN, inglês completo)
```

Interface MarketConfig define: phonePrefix, currency, smsProvider, mapsProvider, defaultCenter, emergencyNumber, privacyRegulation, theme.accentColor, features{inAppPayments, gdprConsent, googleMaps...}

Detecção automática: locale do dispositivo → mercado.
Fallback: Angola (pt-AO).

```
CONVENÇÕES DE CÓDIGO (obrigatórias)
- Todos os textos da UI via useT("namespace")
  const { t } = useT("ride");
  t("request_ride") → "Pedir Boleia"(AO) ou "Pedir Carona"(BR)
- Config do mercado via useMarket()
  const { config, hasFeature, formatCurrency } = useMarket();
- Feature flags: {hasFeature("inAppPayments") && <PagamentoBtn/>}
- Número emergência: config.emergencyNumber (nunca hardcoded)
- Provider SMS: config.smsProvider (nunca hardcoded)
- Comentários: Português de Portugal
- TypeScript strict – sem "any"
- Cada ficheiro: // Ficheiro: X | Função: Y
```

PALETA

```
Navy #0D1F38 · Amber #D97706 · Verde #10B981 · Vermelho SOS #EF4444
Surface #F2F5FB · Texto #0D1F38 · Texto2 #4B5775 · Bordo #E2E8F2
```

P1 **SETUP INICIAL DO PROJECTO**

Semana 1 · Dia 1-2

ACTUALIZADO

O que mudou vs v1: Adicionadas dependências `il8n`, `marketStore` (Zustand), e ficheiro `theme.ts` actualizado com suporte a `accentColor` dinâmico por mercado.

```
+ package.json: + il8next, react-il8next, expo-localization
+ src/store/marketStore.ts (Zustand) - config: MarketConfig, isLoading: bool
+ src/constants/theme.ts - accentColor dinâmico via marketStore
+ src/types/index.ts - + MarketCode, MarketConfig, PaymentMethod
```

PROMPT 1 – Setup Inicial · Cole no Claude Project

Cria a estrutura base do projecto RideFriend v2 (multi-mercado).

1. package.json
Dependências obrigatórias:
expo@51, react-native@0.74, typescript,
@react-navigation/native, @react-navigation/bottom-tabs,
@react-navigation/stack, react-native-screens,
react-native-safe-area-context, @supabase/supabase-js,
react-native-maps, expo-location, expo-notifications,
expo-secure-store, zustand, react-native-mmkv,
@tanstack/react-query, react-native-gesture-handler,
react-native-reanimated, expo-image-picker,
react-native-phone-number-input,
il8next, react-il8next, expo-localization ← NOVO v2
2. tsconfig.json – strict + aliases:
@components, @hooks, @services, @store, @types,
@utils, @constants, @config, @il8n ← @config e @il8n novos
3. src/types/index.ts
Interfaces: User, Contact, Location, Ride, Rating, Notification
Types: RideStatus, ContactGroup, LocationMode
Interfaces derivadas: NearbyDriver, NearbyPassenger
NOVO: MarketCode, MarketConfig, PaymentMethod ← NOVO v2
4. src/constants/theme.ts
Cores base: navy #0D1F38, amber #D97706, green #10B981, red #EF4444
Função getTheme(accentColor?: string): Theme
– accentColor vem da config do mercado activo ← NOVO v2
Tipografia, spacing, radius, sombras (igual v1)
5. src/services/supabase.ts – cliente Supabase + helpers
6. src/store/authStore.ts – user, session, isLoading (igual v1)
7. src/store/marketStore.ts ← NOVO v2
Estado: config: MarketConfig | null, isLoading: boolean
Acções: setMarket(config), reset()
Persistência: último market code em AsyncStorage

```
8. src/store/locationStore.ts - myLocation, nearbyDrivers (igual v1)

9. src/navigation/RootNavigator.tsx
  Igual v1 + adicionar MarketSelect no fluxo de onboarding

10. .env.example - adicionar NOVO v2:
    AFRICAS_TALKING_API_KEY=
    AFRICAS_TALKING_USERNAME=
    TWILIO_ACCOUNT_SID=
    TWILIO_AUTH_TOKEN=
    TWILIO_FROM_NUMBER=
    TERMII_API_KEY=
    GOOGLE_MAPS_API_KEY= (opcional, só BR e PT)

11. app.json - igual v1
```

P2

BASE DE DADOS SUPABASE

Semana 1 · Dia 2-3

ACTUALIZADO

- + Coluna market_code VARCHAR(2) em: users, rides, notifications
- + Nova tabela: bus_stops (paragens por mercado e cidade)
- + Função SQL: nearest_stop(lat, lng, market, radius_m)
- + Seed expandido: paragens de Luanda, Maputo, São Paulo, Lisboa, Lagos
- + RLS: bus_stops visíveis a todos os autenticados

PROMPT 2 – SQL Supabase · Cole no SQL Editor do Supabase

Gera o SQL completo de setup do RideFriend v2 para o Supabase.

1. EXTENSÕES: uuid-oss, postgis, pg_trgm
2. TIPOS ENUM: contact_group, location_mode, ride_status (igual v1)
3. TABELAS – igual v1 com estas adições:
 - users: + coluna market_code VARCHAR(2) DEFAULT "ao"
 - rides: + coluna market_code VARCHAR(2)
 - notifications: + coluna market_code VARCHAR(2)
4. NOVA TABELA: bus_stops ← NOVO v2
 - id UUID PK, market_code VARCHAR(2) NOT NULL,
 - city VARCHAR(50), name VARCHAR(100),
 - lat DECIMAL(10,8), lng DECIMAL(11,8),
 - is_major BOOLEAN DEFAULT false
 - INDEX GIST em (lat, lng) via PostGIS
 - RLS: SELECT para todos os utilizadores autenticados
5. ÍNDICES – igual v1
6. RLS – igual v1 (users, contacts, locations, rides, ratings, sos)
7. FUNÇÃO: nearby_drivers(user_id, radius_km) – igual v1
8. NOVA FUNÇÃO: nearest_stop(p_lat, p_lng, p_market, p_radius_m)
 - Retorna a paragem mais próxima do utilizador no seu mercado
 - Usa ST_Distance com PostGIS para cálculo exacto
 - Retorna: { id, name, lat, lng, distance_m }
9. TRIGGERS: rating_avg, updated_at – igual v1
10. SEED – EXPANDIDO v2:
 - Angola (Luanda)
 - ("ao","Luanda","Av. Lenine · Paragem Central",-8.8147,13.2302,true)
 - ("ao","Luanda","Talatona · Shopping",-8.9186,13.1847,true)
 - ("ao","Luanda","Viana · Paragem Principal",-8.9039,13.3728,true)
 - ("ao","Luanda","Largo do Kinaxixe",-8.8200,13.2350,true)
 - Moçambique (Maputo)
 - ("mz","Maputo","Terminal Museu",-25.9692,32.5732,true)
 - ("mz","Maputo","Mercado Central",-25.9653,32.5791,true)
 - Brasil (São Paulo)

```
("br","São Paulo","Terminal Jabaquara",-23.6427,-46.6534,true)
("br","São Paulo","Av. Paulista c/ Consolação",-23.5614,-46.6565,true)
-- Portugal (Lisboa)
("pt","Lisboa","Marquês de Pombal",38.7252,-9.1500,true)
("pt","Lisboa","Oriente",38.7681,-9.0990,true)
-- Nigéria (Lagos)
("ng","Lagos","Oshodi Terminal",6.5480,3.3540,true)
("ng","Lagos","CMS Bus Stop",6.4536,3.3947,true)

8 utilizadores seed com market_code e coordenadas correctas
(igual v1 mas adicionar campo market_code em cada utilizador)
```


P3

AUTENTICAÇÃO OTP POR SMS

Semana 2 · Dia 1-2

ACTUALIZADO

- + `auth.service.ts`: `smsProvider` vem de `config.smsProvider` (não hardcoded)
- + `PhoneInputScreen`: `phonePlaceholder` e `phonePrefix` do `market config`
- + `OnboardingScreen`: passo 1 = selecção de mercado, passo 2 = perfil
- + Labels adaptados ao mercado (Bairro / Localidade / Area)

PROMPT 3 – Autenticação · Cole no Claude Project

Cria o fluxo completo de autenticação do RideFriend v2.

1. `src/services/auth.service.ts`
 - `sendOTP(phone: string): Promise<void>`
 - Lê `config` do mercado activo via `useMarketStore.getState().config` ← NOVO v2
 - Usa `config.smsProvider` para seleccionar provider ← NOVO v2
 - Importa e chama `sms.service.ts` (criado no Prompt L3)
 - Erros em português (ou inglês para mercado ng)
 - `verifyOTP`, `signOut`, `refreshSession` – igual v1
2. `src/screens/auth/PhoneInputScreen.tsx`
 - `phonePrefix` e `phonePlaceholder` vêm de `useMarket().config` ← NOVO v2
 - Picker de país pré-selecciona o mercado activo
 - Exemplo AO: prefixo "+244", placeholder "9XX XXX XXX"
 - Exemplo BR: prefixo "+55", placeholder "(11) 9XXXX-XXXX"
 - Exemplo NG: prefixo "+234", placeholder "080X XXX XXXX"
 - Validação usa `config.phoneMinDigits` e `config.phoneMaxDigits`
3. `src/screens/auth/OTPVerifyScreen.tsx` – igual v1
4. `src/screens/auth/OnboardingScreen.tsx` ← ACTUALIZADO v2
 - PASSO 1: Selecção de mercado (`MarketSelectScreen` embutido)
 - Mostra mercado detectado automaticamente
 - "Detectámos que estás em Angola. Correcto?"
 - Opção de mudar com lista dos 6 mercados
 - PASSO 2: Formulário de perfil (igual v1 mas labels do mercado)
 - Label "Bairro/Zona":
 - AO/MZ/BR/CV: "Bairro" | PT: "Localidade" | NG: "Area"
 - Guarda `market_code` em tabela `users` ao criar perfil ← NOVO v2
5. `src/screens/auth/MarketSelectScreen.tsx` ← NOVO v2
 - Lista 6 mercados: bandeira, nome, cidades principais
 - Mercado activo com borda de destaque
 - `onSelect`: `setMarket` + `loadMarketConfig` + actualizar `i18n`
6. `src/hooks/useAuth.ts` – igual v1

P4 LOCALIZAÇÃO EM TEMPO REAL

Semana 2 · Dia 3-5

ACTUALIZADO

- + geocoding.service.ts: usa config.geocodingProvider (nominatim ou google)
- + getNearestStop(): consulta tabela bus_stops filtrada por market_code
- + reverseGeocode retorna shortName adaptado à estrutura de endereços local

PROMPT 4 – Localização · Cole no Claude Project

Cria o motor de localização do RideFriend v2.

1. src/utils/geo.ts – igual v1
haversineDistance, estimateETA, isPointNearRoute,
formatDistance, formatETA
2. src/services/geocoding.service.ts ← ACTUALIZADO v2
reverseGeocode(lat, lng): Promise<GeoResult>
– Lê config.geocodingProvider do marketStore
– Se "nominatim": usa Nominatim OpenStreetMap
Accept-Language: config.locale (ex: "pt-AO", "en-NG")
– Se "google": usa Google Geocoding API com GOOGLE MAPS API KEY
– GeoResult: { displayName, shortName, lat, lng }
– shortName adapta-se: bairro+rua (AO/MZ) ou bairro (BR) ou rua (PT)

searchAddress(query, nearLat, nearLng): Promise<GeoResult[]>
– Autocomplete com Nominatim ou Google conforme provider
– viewport centrado nas coordenadas do utilizador

getNearestStop(lat, lng): Promise<BusStop | null> ← NOVO v2
– Chama função Supabase RPC "nearest_stop"
– Passa: p_lat, p_lng, p_market (do marketStore), p_radius_m: 800
– Retorna { name, lat, lng, distance_m } ou null
3. src/services/location.service.ts – igual v1
startTracking, stopTracking, publishLocation, requestPermissions
4. src/services/realtime.service.ts – igual v1
5. src/hooks/useLocation.ts ← ACTUALIZADO v2
– Após obter localização, chama getNearestStop()
– Expõe: myLocation, nearestStop, isTracking, permissionStatus
– nearestStop.name é mostrado no HeroLocationCard
6. src/hooks/useNearbyContacts.ts – igual v1
7. src/utils/__tests__/geo.test.ts – igual v1
+ adicionar teste de getNearestStop com mock Supabase

PL1

i18n + MARKET CONFIG

Semana 2 · Dia 5

NOVO

Prompt completamente novo. Implementa toda a arquitetura de localização descrita no documento RideFriend_MultiMercado.docx. Deve ser executado ANTES do Prompt 5.

PROMPT L1 – i18n + MarketConfig · Cole no Claude Project

Implementa a arquitetura completa de multi-mercado.
Suportar: ao, mz, br, cv, pt, ng. Uma única codebase.

Instalar: `npm install i18next react-i18next expo-localization`

1. `src/config/markets/types.ts`

Interface MarketConfig:

```
code: MarketCode (ao|mz|br|cv|pt|ng)
name, locale, timezone
phonePrefix, phoneMinDigits, phoneMaxDigits, phonePlaceholder
smsProvider: "africas_talking"|"twilio"|"termii"
mapsProvider: "osm"|"google"
defaultCenter: { lat, lng }, defaultZoom
geocodingProvider: "nominatim"|"google"
currency: { code, symbol, decimals }
emergencyNumber: string
privacyRegulation: "lgpd"|"rgpd"|"none"
dataResidency: "local"|"eu"|"any"
paymentMethods?: PaymentMethod[]
features: { inAppPayments, gdprConsent, googleMaps,
            backgroundLocation, vehicleVerification }
theme: { accentColor, accentColorLight }
supportWhatsApp?: string
```

2. `src/config/markets/ao.config.ts` (base)

```
+244, AOA, Africa/Luanda, africas_talking, osm, nominatim,
emergência 113, features todos false excepto backgroundLocation
theme: accentColor #10B981
```

3. `src/config/markets/mz.config.ts`

```
+258, MZN, Africa/Maputo, africas_talking, emergência 119
```

4. `src/config/markets/br.config.ts`

```
+55, BRL, America/Sao_Paulo, twilio, google, lgpd,
inAppPayments: true, vehicleVerification: true, emergência 190
paymentMethods: [{ id: "pix", name: "PIX" }]
```

5. `src/config/markets/cv.config.ts`

```
+238, CVE, Atlantic/Cape_Verde, africas_talking, emergência 132
```

6. `src/config/markets/pt.config.ts`

```
+351, EUR, Europe/Lisbon, twilio, google, rgpd,
dataResidency: "eu", gdprConsent: true, emergência 112
paymentMethods: [{ id: "mbway", name: "MB Way" }]
```

```
7. src/config/markets/ng.config.ts
+234, NGN, Africa/Lagos, termii, osm, en-NG, emergência 199
vehicleVerification: true

8. src/config/markets/index.ts
loadMarketConfig(code: MarketCode): Promise<MarketConfig>

9. src/il8n/detector.ts
detectMarket(): Promise<MarketCode>
1º: AsyncStorage "rf_market"
2º: Localization.locale do dispositivo
3º: fallback "ao"
setMarket(code): Promise<void>

10. src/il8n/locales/pt-AO/common.json (textos gerais)
src/il8n/locales/pt-AO/ride.json (boleia, paragem, motorista)
src/il8n/locales/pt-AO/auth.json (login, registo)
src/il8n/locales/pt-AO/errors.json (erros)

11. src/il8n/locales/pt-BR/ride.json (só diferenças)
carona, ônibus, "tô aqui!", "chegando", "motorista"
src/il8n/locales/pt-BR/auth.json

12. src/il8n/locales/pt-PT/ride.json (só: condutor, condutores)

13. src/il8n/locales/en-NG/ (inglês completo: common, ride, auth)

14. src/il8n/index.ts
initIl8n(): Promise<void>
il8next + initReactIl8next
fallbackLng: "pt-AO" (herança para pt-MZ, pt-CV, pt-PT)
namespaces: common, ride, auth, errors

15. src/hooks/useT.ts
Wrapper useTranslation com namespace tipado
const { t } = useT("ride")

16. src/hooks/useMarket.ts
const { config, formatCurrency, validatePhone,
formatPhone, hasFeature } = useMarket()

17. Integrar initIl8n() e loadMarketConfig() no arranque da app
(antes de renderizar qualquer ecrã)
```

PL2

MARKET ONBOARDING + FEATURE
FLAGS

Semana 3 · Dia 1

NOVO

PROMPT L2 – Market Onboarding · Cole no Claude Project

Integra selecção de mercado e feature flags na app.

1. Modificar OnboardingScreen para 2 passos:
Passo 1: Selecção de mercado
 - Lista 6 mercados com bandeira + nome + cidades
 - Destaca mercado detectado automaticamente
 - "Detectámos que estás em Angola. Correcto?"Passo 2: Formulário de perfil (adapta labels ao mercado)
2. Adicionar opção nas Definições: "Mudar de País/Mercado"
 - Abre MarketSelectScreen
 - Ao confirmar: reconfigura i18n + theme + marketStore
3. Ecrã de consentimento RGPD (só Portugal):
{hasFeature("gdprConsent")} && <GdprConsentScreen/>
 - Aparece no primeiro launch se mercado for "pt"
 - Opções: Aceitar, Recusar, Ver política completa
 - Guardado em AsyncStorage: "rf gdpr accepted"
4. Ecrã de consentimento LGPD (só Brasil):
Similar ao RGPD mas com texto adaptado à lei brasileira
Aparece automaticamente se mercado for "br"
5. Botão PIX no ecrã de confirmação de boleia (só Brasil):
{hasFeature("inAppPayments")} && <PixPaymentButton/>
 - Placeholder: "Pagar com PIX (em breve)"
6. Verificação de matrícula no onboarding (BR/PT/NG):
{hasFeature("vehicleVerification")} && <PlateVerifyField/>
 - Campo extra no formulário de motorista

PL3

SMS MULTI-PROVIDER

Semana 3 · Dia 1

NOVO

PROMPT L3 – SMS Multi-Provider · Cole no Claude Project

Implementa o dispatcher de SMS que selecciona o provider correcto.

1. src/services/sms.service.ts
sendOtpSms(phone: string, otpCode: string): Promise<void>
 - Lê config.smsProvider do marketStore
 - Mensagem adaptada ao locale:
 - pt-AO/MZ/CV/PT: "O teu código RideFriend é: XXXX. Válido 10 min."
 - pt-BR: "Seu código RideFriend é: XXXX. Válido por 10 min."
 - en-NG: "Your RideFriend code is: XXXX. Valid for 10 minutes."
 - Switch: "africas_talking" | "twilio" | "termii"
sendAfricasTalking(phone, message):
 - POST https://api.africastalking.com/version1/messaging
 - Headers: apiKey, Content-Type: application/x-www-form-urlencoded
 - Body: username, to, message
sendTwilio(phone, message):
 - POST https://api.twilio.com/2010-04-01/Accounts/{SID}/Messages.json
 - Auth Basic (SID:TOKEN em base64)
sendTermii(phone, message):
 - POST https://api.ng.termii.com/api/sms/send
 - JSON: { api_key, to, from: "RideFriend", sms, type: "plain" }
2. backend/src/services/sms.backend.service.ts
Versão para o servidor Node.js – mesma lógica mas com env vars do servidor (não expostas no cliente)
3. Testes: src/services/__tests__/sms.service.test.ts
 - Mock de cada provider
 - Testar que o provider correcto é chamado por mercado
 - Testar formato da mensagem em cada idioma

P5

ECRÃ PRINCIPAL — PASSAGEIRO

Semana 3 · Dia 2-3

ACTUALIZADO

- + Todos os textos da UI via `useT("ride")` – nunca strings hardcoded
- + `HeroLocationCard`: nome da paragem via `nearestStop (getNearestStop)`
- + `AlertButton`: texto do botão via `t("im_here_btn")`
- + `DriverCard`: etiquetas de relação via `t("rel_family")` etc.

PROMPT 5 – Ecrã Passageiro v2 · Cole no Claude Project

Cria o ecrã principal do modo Passageiro com suporte multi-mercado.

REGRA OBRIGATÓRIA: zero strings hardcoded na UI.

Todos os textos via: `const { t } = useT("ride")`

1. `src/components/ui/AvatarBadge.tsx` – igual v1
2. `src/components/ui/ETABadge.tsx` – igual v1
3. `src/components/ui/ApproachBar.tsx` – igual v1
4. `src/components/home/DriverCard.tsx` ← ACTUALIZADO v2
Textos via `useT()`:
 - `t("nearby_drivers")` no título da secção
 - `t("request_ride")` no botão primário
 - `t("rel_family")` / `t("rel_friend")` / `t("rel_colleague")` / `t("rel_neighbour")`
 - `t("eta_min", { n: driver.eta })` para o ETA
 - `t("distance_km", { n: driver.distance })` para a distância
5. `src/components/home/HeroLocationCard.tsx` ← ACTUALIZADO v2
 - Nome da paragem: `nearestStop?.name ?? t("detecting_location")`
(`nearestStop` vem do hook `useLocation`)
 - `t("waiting_since", { time: waitTime })`
 - `t("nearby_count", { count: nearbyDrivers.length })`
6. `src/components/home/AlertButton.tsx` ← ACTUALIZADO v2
 - Texto normal: `t("im_here_btn")`
 - AO: "Estou Aqui! · Avisar Rede"
 - BR: "Tô Aqui! · Avisar a Galera"
 - NG: "I'm Here! · Alert My Network"
 - Texto enviado: `t("im_here_sent", { count })`
7. `src/screens/home/PassengerHomeScreen.tsx` ← ACTUALIZADO v2
 - Igual v1 mas todos os textos via `useT()`
 - Estado vazio: `t("no_drivers")`
 - SOS: usa `config.emergencyNumber` (nunca "113" hardcoded)
8. `src/components/home/RideConfirmBottomSheet.tsx` ← ACTUALIZADO v2
 - Mensagem pré-preenchida usa `t("ride")` + `nearestStop.name`
 - AO: "Estou na [paragem]. Podes dar boleia até [zona]?"
 - BR: "Tô no [ponto]. Pode me dar uma carona até [bairro]?"
 - NG: "I'm at [stop]. Can you give me a lift to [area]?"
9. `src/hooks/useRideRequest.ts` – igual v1

P6

ECRÃ MODO MOTORISTA

Semana 3 · Dia 4-5

IGUAL

Sem alterações vs v1. O modo motorista não tem textos específicos por mercado além dos cobertos pelo i18n global. Executar o prompt v1 original.

PROMPT 6 – Modo Motorista · Cole no Claude Project (igual v1)

Cria o ecrã e lógica do Modo Motorista.

(Conteúdo idêntico ao Prompt 6 da v1 – ver documento v1)

Notas v2 (aplicar durante a implementação):

- Textos do DriverStatusCard via useT("ride")
- t("offer_ride") no botão de oferecer boleia
- t("driver label") no título do modo
- t("at_stop") no badge de passageiro na paragem
- Resto igual ao v1

P7

MAPA INTERACTIVO

Semana 4 · Dia 1-2

IGUAL

Sem alterações estruturais vs v1. O provider de mapas (OSM ou Google) já é definido pelo market config e pelo hook useMarket(). Sem código específico adicional necessário.

PROMPT 7 – Mapa · Cole no Claude Project (com nota v2)

Cria o ecrã de mapa interativo em tempo real.
(Conteúdo idêntico ao Prompt 7 da v1)

Nota OBRIGATÓRIA v2 – adicionar ao MapScreen:

```
const { config } = useMarket();  
const mapProvider = config.features.googleMaps  
  ? PROVIDER_GOOGLE  
  : PROVIDER_DEFAULT;
```

```
<MapView provider={mapProvider} ...>
```

Para OSM (ao/mz/cv/ng):

```
urlTemplate: "https://tile.openstreetmap.org/{z}/{x}/{y}.png"
```

Para Google (br/pt):

Não precisa urlTemplate – Google Maps é o default.

Requer GOOGLE MAPS API KEY no app.json e no .env

P8 NOTIFICAÇÕES + SOS

Semana 4 · Dia 3-5

ACTUALIZADO

- + SOS: emergencyNumber vem de config.emergencyNumber (nunca hardcoded)
- + Mensagens push em múltiplos idiomas via i18n no servidor
- + SOSConfirmSheet mostra número correcto por mercado

PROMPT 8 – Notificações + SOS v2 · Cole no Claude Project

Cria o sistema de notificações push e módulo SOS com multi-mercado.

1. src/services/notifications.service.ts – igual v1
 Handler por tipo: driver_approaching, ride_request, ride_accepted, sos_alert, contact_waiting
2. backend/src/services/push.service.ts ← ACTUALIZADO v2
 sendPushToUser(userId, title, body, data)
 - Busca market_code do utilizador na tabela users
 - title e body devem ser enviados já no idioma correcto (quem chama esta função passa os textos traduzidos)
 sendPushToContacts(userId, radius_km, type)
 - type determina qual mensagem enviar
 - Busca market_code de cada contacto para personalizar texto
 - Exemplo "contact_waiting":
 AO: "[Nome] está na paragem. Podes dar boleia?"
 BR: "[Nome] está no ponto. Pode dar uma carona?"
 NG: "[Nome] is at the bus stop. Can you give a lift?"
 sendSosAlert(userId, lat, lng) ← ACTUALIZADO v2
 - Usa SMS backup via sms.backend.service.ts (multi-provider)
 - Número da Polícia vem de config do mercado (não hardcoded)
3. src/components/sos/SOSConfirmSheet.tsx ← ACTUALIZADO v2
 - Número de emergência: config.emergencyNumber
 - AO: "Polícia Nacional · 113"
 - BR: "Polícia Militar · 190"
 - PT: "Número de Emergência · 112"
 - NG: "Emergency · 199"
 - Texto via useT(): t("sos_confirm_message")
4. src/components/ui/SOSButton.tsx – igual v1
5. src/screens/profile/EmergencyContactScreen.tsx – igual v1

P9

BACKEND + DEPLOY HOSTINGER

Semana 5

ACTUALIZADO

```
+ app.ts: middleware de detecção de market_code em cada request
+ routes: endpoint POST /market/detect para o cliente
+ deploy.sh: variáveis de ambiente para AfricasTalking + Twilio + Termii
+ ecosystem.config.js: env vars multi-provider
+ .env.example: expandido com todos os providers
```

PROMPT 9 – Backend + Deploy v2 · Cole no Claude Project

Cria o backend completo e scripts de deploy para o VPS Hostinger.

1. backend/src/app.ts ← ACTUALIZADO v2
 - Igual v1 (cors, helmet, rate-limit, Morgan, JWT)
 - Adicionar middleware: extrai market_code do JWT ou header X-Market-Code e injeta em req.marketCode
 - Health check: GET /health retorna também os mercados activos
2. backend/src/routes/ – igual v1 + novo:
 - GET /market/detect → detecta mercado por IP geolocalização (usa ip-api.com para país → market_code)
 - GET /market/stops?market=ao&lat=&lng= → paragens próximas
3. backend/src/controllers/ – igual v1
4. backend/ecosystem.config.js ← ACTUALIZADO v2
 - env: adicionar todas as variáveis dos providers SMS:
 - AT_API_KEY,
 - AT_USERNAME,
 - TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM_NUMBER,
 - TERMII_API_KEY,
 - GOOGLE_MAPS_API_KEY
5. nginx/ridefriend.conf – igual v1
6. scripts/deploy.sh ← ACTUALIZADO v2
 - Igual v1 + ao criar o ficheiro .env no servidor, o script pergunta e preenche cada provider:
 - "Tens conta AfricasTalking? (s/n)"
 - "Tens conta Twilio? (s/n)"
 - "Tens conta Termii (Nigéria)? (s/n)"
7. .github/workflows/deploy.yml – igual v1
8. .env.example ← EXPANDIDO v2
 - # Supabase
 - SUPABASE_URL=
 - SUPABASE_SERVICE_ROLE_KEY=
 - # SMS – AfricasTalking (Angola, Moçambique, Cabo Verde, Nigéria)
 - AT_API_KEY=
 - AT_USERNAME=
 - # SMS – Twilio (Brasil, Portugal)
 - TWILIO_ACCOUNT_SID=

```
TWILIO_AUTH_TOKEN=
TWILIO_FROM_NUMBER=
# SMS - Termii (Nigéria alternativo)
TERMII_API_KEY=
# Mapas
GOOGLE_MAPS_API_KEY= # opcional, só BR e PT
# Segurança
JWT_SECRET=
NODE_ENV=production
```

P10 REDE, PERFIL E HISTÓRICO

Semana 6

ACTUALIZADO

- + SettingsScreen: secção "Mercado / País" com MarketSelectScreen
- + ProfileScreen: mostra badge do mercado activo no perfil
- + Todos os textos via useT() nos novos ecrãs

PROMPT 10 – Rede + Perfil v2 · Cole no Claude Project

Cria os ecrãs de rede, perfil e definições com multi-mercado.

1. src/screens/network/NetworkScreen.tsx – igual v1
(textos via useT("common"))
2. src/components/network/AddContactSheet.tsx – igual v1
3. src/screens/profile/ProfileScreen.tsx ← ACTUALIZADO v2
 - Mostrar badge do mercado activo no hero card:
bandeira + nome do mercado (ex: 🇦🇴 Angola)
 - Estatísticas: igual v1
4. src/components/profile/RideHistoryItem.tsx – igual v1
(textos via useT("ride"))
5. src/components/profile/RatingBottomSheet.tsx – igual v1
Sugestões rápidas via i18n:
t("rating_punctual"), t("rating_friendly"), t("rating_clean")
6. src/screens/settings/SettingsScreen.tsx ← ACTUALIZADO v2
Adicionar secção "Mercado / País":
 - Mostra mercado actual: bandeira + nome
 - Botão "Mudar de País" → MarketSelectScreen
 - Aviso após mudar: "App reconfigurada para [País]"Resto igual v1 (notificações, privacidade, viatura, emergência)
7. src/screens/auth/MarketSelectScreen.tsx
(já criado no Prompt L2 – reutilizar aqui)

P11**TESTES + PREPARAÇÃO BETA**

Semana 7

ACTUALIZADO

- + Testes para `detectMarket()`, `loadMarketConfig()`, `sms.service dispatcher`
- + Teste de RLS: utilizador AO não vê `bus_stops` de "ng"
- + Seed expandido: utilizadores em múltiplos mercados
- + docs/SETUP.md: secção sobre configuração de cada SMS provider

PROMPT 11 – Testes + Beta v2 · Cole no Claude Project

Prepara o projecto para os primeiros utilizadores reais.

1. Testes unitários Jest – igual v1 + NOVO:

- ```
src/i18n/__tests__/detector.test.ts
```
- `detectMarket()` com locale "pt-BR" → retorna "br"
  - `detectMarket()` com locale "pt-AO" → retorna "ao"
  - `detectMarket()` com locale "en-NG" → retorna "ng"
  - `detectMarket()` com `AsyncStorage` preenchido → usa guardado

- ```
src/config/__tests__/markets.test.ts
```
- `loadMarketConfig("ao")` tem `phonePrefix "+244"`
 - `loadMarketConfig("br")` tem `inAppPayments: true`
 - `loadMarketConfig("pt")` tem `gdprConsent: true`
 - `loadMarketConfig("ng")` tem locale "en-NG"

- ```
src/services/__tests__/sms.service.test.ts
```
- Mock market "ao" → chama `sendAfricasTalking`
  - Mock market "br" → chama `sendTwilio`
  - Mock market "ng" → chama `sendTermii`
  - Mensagem em pt-BR usa "Seu código" (não "O teu")

2. Testes integração supertest – igual v1 + NOVO:

- ```
GET /market/stops?market=ao&lat=-8.8147&lng=13.2302
```
- deve retornar paragens de Luanda
- ```
GET /market/stops?market=br&lat=-23.5505&lng=-46.6333
```
- deve retornar paragens de São Paulo

3. supabase/seed.sql ← EXPANDIDO v2

Utilizadores em múltiplos mercados:

- 4 utilizadores Angola (Luanda)
- 2 utilizadores Moçambique (Maputo)
- 2 utilizadores Brasil (São Paulo)

Com `market_code` preenchido em cada registo

Paragens de cada cidade já incluídas no Prompt 2

4. docs/SETUP.md ← EXPANDIDO v2

Nova secção "Configuração de SMS por mercado":

- AfricasTalking: criar conta em `africastalking.com`, activar Sandbox para testes, obter API key
- Twilio: criar conta em `twilio.com`, verificar número, obter Account SID e Auth Token
- Termii: criar conta em `termii.com`, obter API key para Nigéria

```
Nova secção "Testar múltiplos mercados em dev":
 await setMarket("br") → simula utilizador Brasil
 await setMarket("ng") → simula utilizador Nigéria
```

5. docs/API.md – igual v1 + documentar:  
GET /market/detect, GET /market/stops

6. Checklist de lançamento v2:

- [ ] Auth OTP funcional com +244 (Angola)
- [ ] Auth OTP funcional com +55 (Brasil)
- [ ] t("request\_ride") retorna "Pedir Boleia" para AO
- [ ] t("request\_ride") retorna "Pedir Carona" para BR
- [ ] t("request\_ride") retorna "Request a Lift" para NG
- [ ] nearestStop() retorna paragem de Luanda para coords AO
- [ ] SOS mostra "113" para AO, "190" para BR, "112" para PT
- [ ] hasFeature("inAppPayments") false para AO, true para BR
- [ ] hasFeature("gdprConsent") false para AO, true para PT
- [ ] Mudar mercado nas Definições actualiza UI imediatamente